C++ contracts with regards to function pointers

Document number	D3250R1
Date	2024-04-22
Reply-to	Peter Bindels <dascandy@gmail.com>
Targeted subgroups	SG21

§ 1 Introduction

The current papers on contracts [1], [2] discuss in detail how the contracts facility inside C++ should work, with the perspective of having a future C++ standard version, and how a code base composed solely of C++26+ code using contracts will work. In this paper, I will be exploring the space where languages meet - both non-C++ languages and codebases written in and compatible with prior revisions of C++. With the approach outlined in this paper, libraries can be used from older c++ code bases, C code bases and other languages with less boilerplate and in many cases without any breaking changes, while still being able to assert contracts on their interface.

This paper constitutes a breaking change on the current MVP as it changes the behavior of function pointer conversion. Specifically, it advocates to make function pointer conversion in a deduced context ill-formed, to reserve this space for a future function-pointers-with-contracts feature. The paper assumes familiarity with P2900.

§ 2 Revision history

R1: Remove discussion on implementation strategy and move to separate paper, focusing this paper on only whether or not a function can be deduced as a function pointer / whether to reserve space for function pointers with contracts.

§ 3 Contracts and function pointers

P2900 leaves open the question whether contracts are checked on the caller side or the callee side. This mostly works (see other paper P33xX for a treatise on implementation details), except for the case of function pointers.

A function pointer is the address of a function, stored as an assignable value. In P2900 so far, a function pointer cannot have contracts specified on them. This restriction initially follows from the idea to make an MVP, and function pointers tread into conversion rules between different sets of contracts, making the current solution a good stopgap for the MVP. This does imply that if a function is invoked through a function pointer, for the contracts to have any effect, they necessarily must be checked callee-side.

Having the check callee-side prevents compilers from entering the function without a callee-side contract check, and prevents compilers from optimizing away such a contract check (for example, if the assertions are provable for the compiler implementation based on the code preceding the function pointer call).

§ 4 Benefits of reserving space for contracts on function pointers

- Being able to annotate a function pointer with a contract would allow a compiler to enter the pointed-to function without doing callee-side checks, and use the same contract annotations to do (or elide) those checks caller-side. Contracts being elided by proof in compilers is a very strong driving force behind wide adoption of contracts to write performant safe code.
- Future contracts on function pointers enable function pointer conversion between different sets of contracts (for example, from a more restrictive to a less restrictive, allowing a function to remain annotated by the stronger contract while being used with a weaker contract).
- The proposed method of reserving space for function-pointers-with-contracts preserves the ability to use function pointers, which is a very important part of interacting with different APIs, such as std::function, and many C-style libraries.

§ 5 Downsides of reserving space for contracts on function pointers

- The address-of operator for a function name would not be implicitly convertible to a function pointer, but only explicitly. Functions cannot then easily be used in a place where their type would be deduced. This means that assigning it to an `auto*` variable will fail to compile, as will use of a function with contracts being passed into a template. This is circumventable with an operator like `operator+(<overload set>)`, which would take the raw function pointer with contract checks (if applicable), but that would be a change to the current proposal too. As the syntax used is the logical syntax for deducing a function pointer-with-contracts, leaving this working will necessarily remove our ability to give that meaning to this construct.

- The space reserved this way for function pointers with contracts may not be a usable space. In P2900 we do make a choice to make contracts not a visible property to reflection, and if function pointers are not trivially assignable and convertible (ie, the restriction suggested above) that is detectable and usable from other functions, metafunctions and reflection to determine whether something has contracts, counter to the direction so far.

§ 6 Changes to P2900

Change in 3.3.5:

The contract assertions on a function have no impact on its type ~~and thus no impact on the type of its address, nor on what types of function pointers that address may be assigned to:~~. The address of a function with contracts is treated as an overload set, making it ambiguous in a deduced context, and selecting the non-contract version in case of a conversion to a type compatible with it.

Add to the code example:

```
using function_type = int(*)(int);
auto *fp2 = f; // ERROR
auto *fp3 = (function_type)f; // OK

template <typename FT>
void g(FT func);
g(f); // ERROR
g((function_type)f); // OK
```

§ 7 Acknowledgements

Major thanks to all of SG21 for the work done on contracts so far; this paper would not be possible without a very well formed P2900 to write it against.

Thanks to Jonas Persson for writing P3221, which provided the direct impetus to write this paper, in a hope to have more C, legacy C++ and other-language compatibility in the MVP at a very minor cost.

Thanks to Corentin Jabot, Miro Knejp, Dawid Pilarski and Tom Honermann for reviewing this paper before publishing, and pointing out some confusing errors before publishing.

Thanks to Ville Voutilainen, Lisa Lippincott and others for additional information during discussions which helped shape the paper into better illuminating both sides of the argument.

§ 8 References

1. P2899 (http://wg21.link/p2899)
2. P2900 (http://wg21.link/p2900)